

SAGA Authorization Study: Paper Outline

Abstract

1 Introduction

As agentic systems increasingly rely on autonomous agent-to-agent communication, they require identity verification, secure communication, and contact policies: authentication answers “who is communicating,” whereas authorization answers “who should be allowed to communicate.” Authenticating a participant and protecting its messages do not establish that a user’s policy permits that participant to communicate.

Authentication and confidentiality verification can establish credential origin and secrecy without establishing authorization correctness. That additional guarantee requires consistency among policy semantics, implementation, and protocol enforcement. We investigate this gap through a SAGA case study, asking whether its authorization pipeline preserves *policy-authorization consistency*: accepted contact must be justified by an applicable decision under the receiving user’s intended policy semantics.

In the diagnostic extension of SAGA’s reference communication model, the existing authentication and token-secrecy queries remain satisfied, while an added authorization correspondence fails. This result identifies a proof coverage gap at token receipt. Separately, the contact policy matcher can allow contact where its documented most-specific-rule semantics require denial. Together, these observations motivate examining the chain from policy specification, through matcher implementation and authorization decision, to protocol capability, with separate evidence for each boundary.

Our analysis has three steps. First, archived direct calls to the matcher show order-dependent decisions: a general permission can override a specific denial. Second, we use ProVerif to analyze the consequences. The formal model abstracts the mismatch using fixed scenario facts: the specification denies contact while the implementation allows it. It does not execute the

Python matcher. A saved trace reaches token issuance (**TokenIssue**) and message acceptance (**ChatAccept**) under these assumptions. Third, we compare Provider and receiver authorization gates: Provider denial prevents material release, whereas receiver denial can prevent issuance after release, under the modeled gate and handoff assumptions. A strict semantic mapping from concrete matcher execution to formal events remains unfinished; service-level end-to-end reproduction is outside the completed evidence.

Our contributions are:

- We identify an authorization enforcement gap between cryptographic verification and policy enforcement in agentic systems through the SAGA case study.
- We demonstrate an implementation-level, order-dependent authorization inconsistency in SAGA’s contact policy matcher.
- We formally analyze how authorization divergence can affect agent communication authorization under explicit modeling assumptions, including the effect of Provider and receiver gate placement.

2 Background

2.1 Agentic Systems and Inter-Agent Communication

Large language model (LLM)-based agents can interact with other agents to coordinate work and delegate tasks on behalf of users. Such interaction allows a task to involve multiple participants, each contributing information or performing part of the work. Communication may proceed autonomously, with limited human involvement in individual exchanges. In this setting, delegation describes assigning work to another agent; the permission to contact that agent is governed by the communication system’s authorization policy.

Inter-agent communication therefore involves both task messages and mechanisms for governing their exchange. Agent identities identify the participants, secure transport protects communication, and authorization policies specify which contacts are permitted. We call the participant requesting contact the *initiating agent* and the participant receiving that request the *receiving agent*.

SAGA provides a concrete setting for these roles. Users register their agents with a Provider that maintains agent contact information and user-defined access-control policies. The Provider assists agents in enforcing these policies, while access-control tokens mediate subsequent communication. This separates the user’s configuration of contact permissions from the agents’ autonomous exchange of task messages.

2.2 Authentication, Authorization, and Confidentiality

Authentication answers “Who is communicating?” It establishes the identity of a participant in a protocol exchange. Identity credentials and checks on protocol messages provide the basis for associating an exchange with that participant.

Authorization answers “Who is allowed to communicate?” It determines whether a participant has permission to perform the requested interaction under an applicable policy. For contact authorization, the relevant decision concerns an initiating agent’s permission to communicate with a receiving agent according to the receiving user’s rules.

Confidentiality answers “Who can learn the exchanged information?” It concerns restricting disclosure of protected information to the parties permitted to learn it within the specified security model. Protected information may include communication content and credentials exchanged by the protocol.

These properties are complementary. Authentication supplies the participant identity to which a policy can apply; authorization determines the permitted interaction; confidentiality protects information exchanged during the protocol. Each describes a distinct aspect of communication security.

2.3 Policy-Based Authorization

Policy-based access control expresses permissions through rules and applies those rules when handling requests. Three concepts organize this process: policy specification, policy evaluation, and enforcement.

Policy specification defines the meaning of the rules, including which subjects and interactions they cover and how applicable rules determine a decision. In SAGA, a receiving user's contact rulebook associates agent-identity patterns with budget values. Pattern matching identifies applicable rules; the policy's rule-selection convention and interpretation of the selected budget determine contact permission.

Policy evaluation applies those semantics to a particular request. Its inputs include the requesting identity, the target, and the applicable policy. The result expresses whether the request is permitted under those inputs. Policy meaning defines the decision to be computed, while the evaluation procedure implements that computation.

Enforcement applies the decision at the relevant protocol boundary. In SAGA's communication flow, the Provider evaluates the receiving user's stored rulebook before releasing the material needed to continue contact. The receiving agent performs policy and credential checks, issues an access-control token, and subsequently checks a presented token when handling a message. These stages distinguish permission to proceed, credential issuance, and message acceptance.

The authorization outcome thus depends on the specified policy semantics, their evaluation for the request, and the application of the resulting decision at the appropriate stages. A policy decision concerns a particular initiating agent and receiving agent; where a credential mediates contact, its use is part of this authorization context. This terminology provides the basis for the policy–authorization consistency requirement developed next.

3 Security Gap and Problem Formulation

3.1 Authentication Is Not Authorization

SAGA uses identity checks and access-control tokens to govern contact between agents. These mechanisms answer different questions. Authentication establishes which participant completed a protocol step; token validation establishes that a presented token satisfies the checks performed by the receiving agent. Authorization additionally asks whether the receiving agent’s user-defined contact policy permits the initiating agent to contact it. A credential can be authentic and valid under the token checks while the decision to make that credential available disagrees with the policy’s intended meaning.

The reference communication model focuses on participant authentication and token secrecy. These guarantees concern the origin of protocol messages and the confidentiality of credentials; they do not, by themselves, establish that a contact complies with the receiving user’s policy. In particular, confidential delivery to an authenticated participant does not establish that the participant is entitled to receive a credential under that policy.

The missing requirement is an authorization basis for accepted contact, preserved through credential issuance and message acceptance. Chapter 4 examines implementation disagreements with policy meaning. This chapter defines the requirement without asserting an end-to-end access violation.

3.2 Policy–Authorization Consistency

We call the end-to-end requirement *policy–authorization consistency*: every accepted access must be traceable to an applicable authorization decision. For every execution and every accepted-access event e , we require

$$\text{AccessAccepted}(e) \implies \exists d. \text{AllowDecision}(d) \wedge \text{Justifies}(d, e).$$

In addition to this end-to-end property, we consider authorization-soundness conditions at intermediate protocol boundaries, including Provider grants and credential issuance. These intermediate events are not themselves instances of `AccessAccepted`.

Here, d is a decision occurring in that execution, and `AllowDecision`(d) means that it grants permission under the intended policy semantics. An implementation’s report of “allow” alone does not establish this predicate. The relation `Justifies`(d, e) requires that the decision precede the accepted access, cover the same subject, target, and, where applicable, operation,

and remain applicable under the relevant policy state. Where a credential mediates access, the decision must also be linked to the grant and credential used for that access. An unrelated earlier allow decision is not a sufficient justification. This is a general requirement whose concrete events and policy-state conditions must be fixed for each formal model.

Let I be an initiating agent, R a receiving agent, and P_R the contact rulebook selected by R 's user. Each rule associates an agent-identity pattern with a budget. We write $\text{Allow}_{\text{spec}}(P_R, I)$ for the decision prescribed by the intended policy semantics, rather than the value returned by any particular implementation of the matcher. In the cases considered here, the unique highest-scoring matching rule determines the budget; a positive budget permits contact, whereas a nonpositive budget or no matching rule does not. Rule selection follows the matcher's documented most-specific-rule contract and its AID specificity score; the positive-budget convention follows the Provider's access check.¹ We initially fix the rulebook throughout an execution and consider either no matching rule or a unique highest-scoring match. Equal-score conflicts require an explicit convention and are outside this initial scope.

We distinguish three analysis-level events. $\text{ProviderGrant}(I, R)$ denotes the Provider's release, through its access interface, of the material needed for I to continue the contact protocol with R . The event $\text{TokenIssued}(R, I, T)$ denotes R 's creation of token T for I and its release for delivery through the protocol; it does not assert that I has received or used T . Finally, $\text{MessageAccepted}(R, I, T, m)$ denotes R 's acceptance of an application message m from I with token T , after the receiving agent's access checks, for application processing. It does not assert the completion of a downstream tool action.

Under this static-policy scope, the intermediate condition at the Provider boundary is

$$\text{ProviderGrant}(I, R) \implies \text{Allow}_{\text{spec}}(P_R, I),$$

whereas a necessary permission condition for end-to-end consistency at the final message-acceptance boundary is

$$\text{MessageAccepted}(R, I, T, m) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

The intermediate condition at token issuance is

$$\text{TokenIssued}(R, I, T) \implies \text{Allow}_{\text{spec}}(P_R, I).$$

¹See `match()` and `aid_specificity()` in `saga/common/contact_policy.py`, and `access()` in `saga/provider/provider.py`.

All three implications range over every occurrence of the respective events and all their arguments. The intermediate conditions are separate obligations, not logical consequences of end-to-end consistency alone. These implications express permission at their respective boundaries; they do not replace the decision-to-access linkage required by **Justifies**. Nor does policy permission alone suffice for access: certificate checks, key availability, token validity, expiry, and quota may still prevent a permitted contact. These conditions address contact permission, not quantitative bounds on credential issuance or message use.

For a policy that can change over time, the applicable policy version and the effect of revocation on existing credentials must additionally be specified. Quota consumption and token expiry likewise require explicit state and validity conditions. The general requirement leaves room for these obligations, but the static permission conditions above do not establish them.

A message-level authorization claim therefore requires a model connecting policy evaluation, credential issuance, and the final message-acceptance event. Chapter 6 develops the corresponding formalization and states its assumptions.

3.3 Authorization Pipeline

The relevant decision is distributed across several stages. The user's rulebook determines an intended answer for I contacting R . The Provider evaluates its stored copy of that rulebook and, if the request passes its checks, returns one-time key material. The initiating and receiving agents then complete the credential exchange; the receiving agent generates a token and later validates a token presented with a message. An authorization argument must connect the same initiating identity and receiving identity across these stages, rather than infer permission solely from the presence of a valid token. The intended progression is

$$P_R \longrightarrow \text{ProviderGrant} \longrightarrow \text{TokenIssued} \longrightarrow \text{MessageAccepted}.$$

The arrows indicate protocol stages and authorization dependencies, not a guarantee that reaching one stage causes the next to succeed.

Each boundary requires separate evidence: a Provider response does not establish token issuance, and token issuance does not establish message acceptance. Later checks and protocol steps may fail.

When $\neg \text{Allow}_{\text{spec}}(P_R, I)$ holds, we distinguish three levels of violation:

1. **Provider decision violation:** $\text{ProviderGrant}(I, R)$ occurs despite the policy denial.

2. **Credential issuance violation:** $\text{TokenIssued}(R, I, T)$ occurs despite the policy denial. Issuance alone does not establish delivery or subsequent use.
3. **End-to-end authorization violation:** $\text{MessageAccepted}(R, I, T, m)$ occurs despite the policy denial. This is acceptance at the receiving agent’s application boundary.

These levels classify possible observations rather than assert results. A sound pipeline must preserve the link to the applicable decision, participants, and credential across stages. Chapter 6 specifies the corresponding events; Chapter 7 evaluates implementation observations and model results separately.

3.4 Threat Model

We consider a normally registered initiating agent controlled by the adversary. It can choose its contact requests and participate in the protocol under its genuine identity and certificate. The receiving user’s rulebook is an existing configuration: the adversary cannot change its contents or rule order, control the Provider or CA, or obtain another honest party’s private keys.

The Provider and receiving agent execute their implementations with the same fixed rulebook. Cryptographic checks are assumed effective. The failure under study is a disagreement between policy meaning and enforcement during ordinary protocol operation, without compromise of either enforcement point.

The adversary’s objective is an execution in which the intended policy denies contact by I , yet the system performs one of the authorization-relevant actions above. At the message level, the violation condition is

$$\text{MessageAccepted}(R, I, T, m) \wedge \neg \text{Allow}_{\text{spec}}(P_R, I).$$

This end-to-end violation is distinct from an erroneous Provider grant or credential issuance. Claiming it requires evidence that the intervening checks and protocol steps reach the receiving agent’s acceptance boundary.

We study this objective under a static policy. Policy updates and revocation, disagreement between stored policy versions, quantitative exhaustion of budgets or quotas, compromise of trusted parties, and arbitrary actions after contact require additional assumptions and are outside this initial question. The distinction between zero, negative, and positive configured budgets remains part of the contact-permission semantics.

4 Authorization Inconsistencies in SAGA

This chapter examines how the reference implementation interprets contact policy at its enforcement points. We distinguish the documented selection of a rule from the interpretation of the selected budget. The analysis concerns the fixed-policy setting of Chapter 3 and is based on source inspection; runtime propagation to the three security boundaries requires separate execution evidence.

4.1 Intended Contact-Policy Semantics

A contact rulebook is represented as an ordered list of rules, each carrying a `pattern` and an integer `budget`. A pattern matches an agent identifier, or AID. For contact from I to R , the relevant object is R 's rulebook evaluated against I 's AID: permission has a direction. Its evaluation does not distinguish individual application commands or tool actions. Throughout this chapter, *policy budget* denotes the `budget` field of a contact rule, not the communication quota associated with an access-control token.

In `contact_policy.py`, `check_rulebook()` checks rule fields and admits integer budgets from -1 upwards. The `match()` function first checks AID syntax and then tests patterns with `fnmatch.fnmatch()`. Its documentation states that the returned budget belongs to the most specific matching pattern. The function `aid_specificity()` supplies the comparison score, using weighted contributions from literals and wildcard constructs in the user and agent-name components. We use this numerical ordering as the documented selection criterion, without assuming that it is a general ordering by containment between pattern languages.

For a well-formed fixed rulebook with a unique highest-scoring match, let b^* denote the budget attached to that rule. For no match, take $b^* = 0$, consistent with the matcher's default result. The policy baseline is

$$\text{Allow}_{\text{spec}}(P_R, I) \iff b^* > 0.$$

The positivity convention comes from the Provider's `/access` handler, which rejects both negative and zero results. Thus the baseline combines a documented rule-selection contract with the Provider's explicit interpretation of the result. It does not assert a separate published policy language or resolve conflicts between equally specific rules. A malformed AID produces a separate error result and is outside the valid-identity cases considered here.

A positive policy decision is only permission to proceed through this check. The Provider additionally uses resource availability and remain-

ing contact allowance, and later protocol stages impose their own checks. Accordingly, this baseline specifies contact eligibility rather than sufficient conditions for token issuance or application-message acceptance.

4.2 Rule-Specificity Selection Flaw

The implementation of `match()` does not maintain the state needed by its documented selection rule. It initializes `best_pattern` to `None` and the return budget to zero. For each matching rule, it compares that rule’s specificity with `aid_specificity(best_pattern)` and updates the budget when the comparison succeeds. It never updates `best_pattern`. Consequently, later rules are compared with the initial sentinel, not with the most specific rule already encountered.

This is more precise than saying that specificity is ignored: the comparison is executed, but its reference value remains wrong. The score of `None` is -1 , whereas the scores computed for well-formed patterns are nonnegative, including zero for the universal wildcard. Therefore every matching rule handled normally by the scorer passes the comparison. For a valid AID and a well-formed rule list, the returned budget is the budget of the last matching rule encountered; with no match it remains zero.

The order involved is the rulebook’s list order, not the order of keys within each rule dictionary. The registration path carries the list in the `contact_rulebook` field, the Provider stores that field, and the Agent loads it into its local state. None of these paths sorts the rules by specificity. The matcher’s ordinary list traversal consequently preserves the distinction between earlier and later rules.

Consider two rules that both match the same valid identity I . Let r_s have a strictly higher specificity score and budget -1 , and let r_g have a lower score and budget 10 . This is an abstract policy example; it does not assume that I can edit R ’s configuration. The documented selection is r_s in either ordering. Inspection of the loop yields the following different implementation results:

Rule order	Intended budget	Returned budget
$[r_s, r_g]$	-1 (deny)	10
$[r_g, r_s]$	-1 (deny)	-1

To compare Boolean decisions, define

$$\text{Allow}_{\text{impl}}(P_R, I) \iff \text{match}(P_R, I) > 0.$$

This is the actual matcher result interpreted by the Provider’s policy check, not the success of the entire endpoint. For the first ordering above,

$$\text{Allow}_{\text{impl}}(P_R, I) \neq \text{Allow}_{\text{spec}}(P_R, I).$$

The same logical rule set thus has an order-dependent implementation result despite an unchanged unique maximum under the documented semantics. Equal scores are unnecessary for this disagreement. The observation establishes a policy-evaluation mismatch; it does not, by itself, establish any of the credential or message events defined in Chapter 3.

4.3 Budget-Semantics Inconsistency

The two enforcement points also differ in how they interpret an identical matcher result. In `provider.py`, `access()` returns a denial for `budget < 0` and separately for `budget == 0`. Only a positive value proceeds to the resource-allocation part of the handler. Writing `ProviderAllow` for this policy check alone gives

$$\text{ProviderAllow}(b) \iff b > 0.$$

This predicate excludes the endpoint’s identity, registration, and resource checks; a true value is not yet a Provider grant.

In `agent.py`, the incoming connection handler evaluates the local rule-book against the initiating AID using `match()`. It rejects only when the returned value is negative. The budget is not tested for equality to zero at this point. For this particular precheck,

$$\text{AgentPrecheckAllow}(b) \iff b \geq 0.$$

Here “allow” means only that this conditional does not reject the connection. It is neither a statement that a token is generated nor a statement that an application message is accepted.

Policy result	Provider policy check	Agent precheck
$b < 0$	Reject	Reject
$b = 0$	Reject	Continue
$b > 0$	Continue	Continue

The difference includes an explicit zero-budget match and the zero returned when no rule matches. At either source of zero, the Provider treats the result as insufficient permission, while the Agent’s precheck does not stop the connection. However, the ordinary fresh-contact path obtains material through the Provider first. Its zero-budget denial remains effective at

that boundary. The receiver-side difference therefore does not independently establish a complete unauthorized execution.

Several distinct quantities must be kept separate when assessing this behavior. The configured rule budget is an input to policy evaluation. The Provider’s **counter** stores a remaining allowance for each initiating AID within a target Agent’s record. A first allocation initializes the entry to the selected budget minus one; subsequent allocations decrement that entry, with a floor of zero. An exhausted entry blocks further allocation. Despite its name, this is remaining contact allowance, not an accumulated count of accepted application messages. Allocation consumes allowance before the later credential exchange has necessarily completed.

Token-use state is maintained separately. The **generate_token()** function sets the token’s **communication_quota** from the configured **Q_MAX** value, not from the matched contact-rule budget. Token validation checks the receiver’s active-token state, expiration, remaining token quota, and recipient access-control key. The conversation loop decrements token quota and also has a separate **MAX_QUERIES** guard. These checks concern token use and local conversation processing. Neither their presence nor their success establishes that the correct policy rule was selected, and the zero-budget comparison cannot be interpreted as a token-quota exhaustion test.

4.4 Cross-Layer Authorization Inconsistency

The findings concern two different relationships. The rule-selection flaw is a *specification-to-implementation inconsistency*: the matcher can return a budget that disagrees with the documented most-specific-rule contract. The zero-budget difference is an *enforcement-to-enforcement inconsistency*: two consumers of the same policy result apply different predicates. In the latter case,

$$\text{ProviderAllow}(0) \neq \text{AgentPrecheckAllow}(0).$$

The second disagreement does not depend on overlapping rules. Conversely, the first can change the sign of the returned budget without involving zero. They should not be assigned the same consequence merely because both concern authorization.

The Provider and receiving Agent both invoke the common matcher. The Agent’s incoming policy check therefore supplies an additional enforcement point, but it does not independently recover the documented rule selection. With the same ordered rulebook and identity, both invocations inherit the same selection error. Agreement between two implementations of a check

is insufficient when both rely on the same incorrect decision procedure. At zero, even their interpretation of that shared result differs.

These observations concern the preservation of policy meaning along the pipeline defined in Chapter 3. The Provider’s successful response supplies contact material, including one public one-time key and its signature, the target Agent’s certificate and public access-control material, and the target user’s certificate. It removes the rulebook and counter from that response. This response is not an access-control token; the receiving Agent generates the token at a later boundary.

The incoming handler performs its policy precheck before credential handling. Subsequent identity, Provider-stamp, signature, and one-time-key checks can still stop a new credential exchange. Their purpose differs from choosing the applicable contact rule. In particular, the Provider stamp authenticates an Agent card constructed at registration, rather than a decision selecting a rule for this contact. Later, `token_is_valid()` checks credential state without re-evaluating rule specificity. Thus successful cryptographic checks cannot be substituted for the missing semantic agreement.

Cryptographic correctness does not repair an incorrect authorization decision. Even assuming that signatures, certificates, token checks, and the authenticated transport work as intended, those mechanisms do not establish that a credential was made available in accordance with the user’s policy. This is a separation of obligations, not a claim that every policy mismatch survives all later checks or that the entire protocol has been exercised.

4.5 Security Consequences

For the rule-selection flaw, source inspection directly establishes that a documented denial can become a positive implementation decision. Because `access()` uses that value for its policy check, the mismatch changes whether execution proceeds towards allocation. The successful allocation branch returns contact material. Conditional on the endpoint’s other checks and ordinary resource requirements succeeding, the mismatch can therefore reach the Provider-grant boundary. This is a conditional control-flow conclusion, not a report of an experimentally observed grant.

In the terminology of Chapter 3, that boundary would constitute a Level 1 violation when

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{ProviderGrant}(I, R).$$

An incorrect positive matcher result alone is not this event: the material

must actually be released. The distinction matters because a policy mismatch can coexist with an unsuccessful endpoint request.

A Level 2 consequence would additionally require

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{TokenIssued}(R, I, T).$$

The shared matcher means that the receiver’s policy precheck need not correct the earlier decision, making propagation relevant to investigate. Nevertheless, token generation occurs after additional checks and depends on valid protocol state. The static mismatch neither satisfies those checks by itself nor records a token being released. It is therefore insufficient as standalone evidence of credential issuance.

Likewise, the Level 3 condition

$$\neg \text{Allow}_{\text{spec}}(P_R, I) \wedge \text{MessageAccepted}(R, I, T, m)$$

requires the receiving application boundary to be reached with an accepted message and valid credential state. Entering a credential-handling branch, generating a token, and accepting a protected message are distinct observations. This chapter establishes no execution exhibiting this final condition.

For the zero-budget discrepancy, the supported conclusion is narrower. The Agent precheck disagrees with the baseline denial at zero, but the Provider’s own decision is unchanged: it rejects that value. The discrepancy alone therefore does not produce a Provider grant on the fresh-contact path. Neither token issuance nor message acceptance follows from the receiver’s failure to reject at this single check. Without additional execution evidence, this finding is a latent authorization inconsistency, not an independently confirmed access violation.

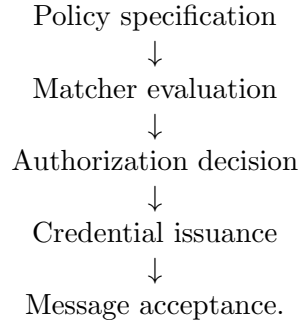
The two findings thus establish concrete policy-evaluation and enforcement disagreements, with different implications for the intermediate boundaries. They show why the authorization-soundness obligations of Chapter 3 cannot be assumed from the implementation’s cryptographic checks. An implementation mismatch is not automatically an end-to-end exploit. Chapter 5 examines concrete execution paths and the evidence needed to determine which, if any, of the Provider-grant, credential-issuance, and message-acceptance boundaries each inconsistency reaches.

5 Attack Path: From Policy Inconsistency to Unauthorized Communication

5.1 Overview

The starting point of this case study is an authorization decision inconsistent with the receiving user’s policy. Following the fixed-policy scope of Chapter 3, the initiating agent has a genuine identity, and the policy is an existing configuration outside its control. The relevant evidence connects two distinct levels: a confirmed matcher-level decision mismatch and a formal analysis of its possible communication consequences. The latter uses an abstraction of the mismatch rather than an execution of the complete service.

The conceptual chain is



These arrows identify dependencies to examine, not implications that every decision necessarily reaches the next stage. In particular, a matcher result, a Provider grant, and an accepted application message require different evidence.

Cryptographic checks remain part of the modeled protocol: the consequence analysis does not disable them or assume that their primitives fail. The saved baseline diagnostic separately preserves its authentication and token-secrecy results. Those results do not automatically transfer to every query in the consequence model. The path studied here concerns the use of an incorrect authorization decision within the modeled protocol.

5.2 Matcher-Level Counterexample

The existing `proofs/test_matcher_bug.py` illustrates the rule-selection inconsistency using this ordered rulebook:

Pattern	Budget	Role
<code>mallory@example.com:*</code>	<code>-1</code>	Specific deny
<code>*</code>	<code>10</code>	General allow

Both rules match the example’s initiating identity. Under the documented most-specific-rule semantics, the first rule determines the result. The expected budget is therefore `-1`, interpreted as denial. The illustrated implementation instead returns `10`, interpreted as permission. Using the decision predicates of Chapters 3 and 4,

$$\begin{aligned}\text{Allow}_{\text{spec}}(P_R, I) &= \text{false}, \\ \text{Allow}_{\text{impl}}(P_R, I) &= \text{true}.\end{aligned}$$

The script is a self-contained comparison of defective and corrected selection logic with a simplified specificity score. It does not import the production matcher. Its role is to expose the decision discrepancy, not to establish general correctness of the corrected procedure.

Direct implementation evidence is provided separately by the archived `policy-matcher-sanity.json`.² That record uses an Alice identity with the same specific-deny/general-allow structure. It reports a valid rulebook, specificity scores of 70 and 0, an expected budget of `-1`, and an actual result of `10`. Reversing the rule order returns `-1`. The identity strings in this record differ from the Mallory illustration and are not presented as the same execution.

Together, these materials establish an implementation-level authorization bypass: the policy enforcement implementation produces an authorization decision inconsistent with the intended semantics. The direct-call record does not exercise Provider requests, token issuance, or message acceptance. The existing rule order is a condition of the example, not an adversarial ability to modify the receiving user’s policy.

5.3 Formalizing the Authorization Divergence

The ProVerif model `agent_communication_authz_chat_turepass.pv`, under `proofs/proverif/`, asks a conditional question: given an authorization divergence, can the modeled protocol reach communication acceptance? It does not execute Python, calculate pattern specificity, or select a rule from a concrete rulebook. Instead, initialization supplies the fixed table facts `ScenarioSpecDeny` and `ScenarioImplAllow` for the scenario identifier `BuggyPolicy` and the initiating agent.

²The file is stored under `proofs/evidence/authz-stage1-20260916T123834944479Z/`.

This is the abstraction boundary. The terms *SpecDecision* and *ImplDecision* are analysis-level labels rather than formal event predicates. They describe the two sides of the authorization inconsistency being modeled. ProVerif models the protocol consequence after the assumed mismatch; it does not compute the matcher result. The two scenario predicates are table facts supplied at initialization. The event **SpecIntendedDeny** records the assumed policy denial. The table fact representing implementation permission allows the modeled enforcement processes to continue.

The participant directions are fixed: **PeerA** represents the receiver R , and **PeerB** represents the initiator I . The relevant observations have the following meanings:

Model observation	Meaning
SpecIntendedDeny (I, R)	Records the intended denial of contact by I with R .
ProviderRelease (R, I)	Records the release of contact material.
TokenIssue (R, I, t)	Records issuance of the wire token t .
ChatAccept (R, I, t, m)	Records R accepting message m from I using t .

The denial event uses the opposite argument order from the release and communication events. This distinction matters when relating the observations to Chapter 3’s **ProviderGrant**, **TokenIssued**, and **MessageAccepted** concepts. That conceptual correspondence is not an implementation-to-model refinement proof.

The model includes a **ProviderReleased** table to represent the hand-off from Provider release to receiver processing. Message acceptance uses an **ActiveToken** association between the receiver, sender, wire token, and sender’s access-control public key. The private channel `tls_chat` abstracts authenticated, confidential transport; it is not a verification of TLS. These assumptions delimit the consequence analysis.

5.4 Formal Reachability Witness

The historical `proofs/evidence/turetrace` contains an existential symbolic reachability witness for **ChatAccept**. The projection below retains the observations relevant to authorization. Line numbers refer to that file, not to the process-position numbers printed inside the trace.

Line	Observation
14, 16	Specification-deny and implementation-allow table facts are inserted.
122	SpecIntendedDeny records the intended denial.
128	ProviderRelease records material release.
168	TokenIssue records the token later used in acceptance.
196	TokenReceive records receipt of that wire token.
202	ChatSend records the message with that token.
208	ChatAccept records acceptance of that message.

In the reachability witness, **TokenIssue** and **ChatAccept** coexist with the assumed policy denial and abstract implementation permission. Denial is a recorded scenario fact, not a blocking guard. The token recorded at issuance also appears in receipt, sending, and message acceptance; the witness therefore extends beyond credential receipt.

ChatAccept means that the receiving agent accepts an application message under a token-mediated communication path, after the checks represented in this model. It does not mean that a production service handled the message or that a downstream tool operation completed. It is also distinct from **Accept**, which denotes token receipt and verification in these models.

The saved result states that `not event(ChatAccept(...))` is false, with a reconstructed witness: message acceptance is existentially reachable. Together with the denial assumption, this supports the abstract authorization consequence, not a failed cryptographic authentication query. The fresh ProVerif 2.05 run reproduces this result and archives the full output.

The witness contains replicated processes and shared token state: the issuing instance at line 168 differs from the accepting instance at line 208. Fresh tokens therefore do not establish session isolation or injective agreement. The historical file omitted the final issuance-correspondence result; the fresh run records **ChatAccept** implying **TokenIssue** as true. This establishes prior issuance with the same receiver, sender, and token within the model, not intended policy permission. No independent **TokenIssue** reachability query exists; the summary distinguishes this absence from issuance observed in the witness.

5.5 Evidence Boundary

Established results. The example script and direct sanity JSON record support the matcher inconsistency. Separately, the consequence model and its witnesses support coexistence of abstract divergence, issuance, and communication acceptance. The fresh run records message acceptance as reach-

able and the issuance correspondence as true. Matcher evidence remains distinct from formal execution evidence.

Claims outside the evidence. These results do not establish a cryptographic primitive failure, certificate forgery, private-key compromise, or arbitrary attacker control. In particular, the initiating participant in the symbolic witness executes the prescribed peer process; the trace does not demonstrate arbitrary application behavior by an external attacker. The static policy and the genuine-identity setting remain those of Chapter 3. The zero-budget discrepancy discussed in Chapter 4 is not an additional completed path in this chapter.

Remaining limitation. The formal model abstracts the matcher result. Direct executable integration with Python and an auditable semantic mapping of identities, policy decisions, and protocol observations remain future work. The `BuggyPolicy` identifier does not encode the concrete rulebook.

The historical consequence trace lacks an independent run manifest for its original execution. The updated evidence package provides reproducible provenance for a fresh execution of the unchanged ProVerif consequence model.³ It records the execution timestamps, Git commit and branch, verifier version, executable and model hashes, command, exit status, raw standard output and error, and a summary extracted from that output. This package documents the new run; it does not retrospectively establish the origin of the historical `turetrace` file.

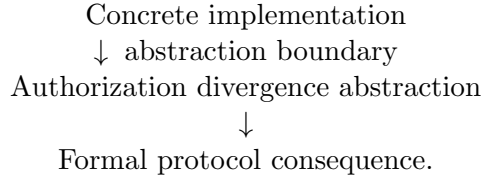
Service-level end-to-end reproduction remains outside the completed evidence. New provenance supplies neither an implementation-to-model mapping nor a production execution, while the implementation-level mismatch remains confirmed.

³New-run evidence: `proofs/evidence/authz-turepass-20260919T090826295518Z/`.

6 Formalization

6.1 Modeling Approach

The formal model captures the consequence of authorization divergence rather than reproducing the matcher implementation. We separate implementation evidence of decision inconsistency from symbolic analysis of what follows when that inconsistency is assumed:



The first arrow is a conceptual connection, not a proved refinement relation. ProVerif neither executes the Python matcher nor discovers its implementation inconsistency. The analysis concerns fixed scenarios and the permission boundaries defined in Chapter 3.

6.2 Protocol Model

The Provider abstracts registration checks, participant authentication, and the authorization-related release of contact material. **PeerA**, the receiver R , issues tokens and receives application messages. **PeerB**, the initiator I , requests a token and sends a message after receiving it. A CA process supplies certificates. Replicated processes use symbolic cryptography and public communication; the private `tls_chat` channel abstracts authenticated, confidential transport.

We distinguish the divergence model, **turepass**, from three independent gate controls. The controls differ only in their preset gate permissions. They use a fixed private token, whereas **turepass** generates fresh tokens. The controls are not verified repairs or a refinement of that model.

6.3 Authorization Events and State

In **turepass**, `ScenarioSpecDeny` and `ScenarioImplAllow` are initialized table facts: analysis-level assumptions representing the observed inconsistency, not matcher execution events. They share a scenario identifier and initiating identity. `SpecIntendedDeny( $I, R$ )` records the assumed denial. The following protocol events use receiver-first arguments; t denotes the wire token.

Event	Observation
ProviderRelease (R, I)	Contact material released.
TokenIssue (R, I, t)	Token issued for the initiator.
TokenReceive (R, I, t)	Initiator received and verified the token.
ChatSend (R, I, t, m)	Initiator sent message m with t .
ChatAccept (R, I, t, m)	Receiver accepted the application message.

The **ProviderReleased** table abstracts a handoff to receiver processing. **ActiveToken** associates receiver, initiator, token, and the sender’s access-control public key. Gate controls additionally require **ProviderAuthorized** for release and **ReceiverAuthorized** for issuance. Receiver processing also requires the handoff state. These tables govern execution; recording an event alone does not grant or deny permission.

Table 1 separates scenario assumptions from observations and verification queries. Divergence is an input to the consequence model; its implementation evidence is independent of the solver results. Gate effects require comparisons between scenarios, not merely the declaration of a release or issuance event.

Claim or assumption	Formal representation and scope
Assumed authorization divergence	ScenarioSpecDeny and ScenarioImplAllow table facts; preset assumptions, not a derived result.
Provider permission affects release	ProviderRelease reachability across gate controls.
Token issuance depends on gate state	TokenIssue reachability across gate controls.
Acceptance requires prior issuance	ChatAccept $\Rightarrow$ TokenIssue ; queried in all four models.
Acceptance has a matching send	ChatAccept $\Rightarrow$ ChatSend ; queried in the three gate controls only.

Table 1: Claim-to-Formalization Mapping. Correspondence arguments and their limitations are specified below.

6.4 Verification Queries

Reachability. The divergence model queries whether **ChatAccept** can occur. The controls separately query **ProviderRelease**, **TokenIssue**, and **ChatAccept** for the fixed peer identities. A false **not event(...)** result

with a reconstructed witness establishes an existential modeled consequence under the scenario assumptions, not a security guarantee. There is no independent **TokenIssue** reachability query in **turepass**.

Correspondence. All four models query

$$\text{ChatAccept}(R, I, t, m) \implies \text{TokenIssue}(R, I, t).$$

The controls also query

$$\text{ChatAccept}(R, I, t, m) \implies \text{ChatSend}(R, I, t, m).$$

Variables are universally quantified. These non-injective correspondences require prior issuance for the same participants and token, or prior sending of the same message, respectively. They do not establish policy permission, unique sessions, or Chapter 3’s full decision-to-access justification.

Gate placement. The controls preset Provider/receiver permissions to allow/allow, deny/allow, and allow/deny. Comparing their three reachability queries locates blocking stages under the shared handoff abstraction.

6.5 Formal Results

The archived ProVerif 2.05 results cover the unchanged inputs; their hashes match the corresponding manifests. In **turepass**, message acceptance is reachable with the assumed denial and abstract implementation permission, and the issuance correspondence is true. Token issuance appears in the witness; it is not reported as a separate query result.

For the controls, the results are:

Provider / Receiver	Release	Issue	Accept
Allow / Allow	Reachable	Reachable	Reachable
Deny / Allow	Unreachable	Unreachable	Unreachable
Allow / Deny	Reachable	Unreachable	Unreachable

Provider denial stops release and downstream events. Receiver denial stops issuance and message acceptance after material release remains possible. Both control correspondences are true in all three scenarios. They are non-vacuous on the allow/allow path; in each denial scenario, acceptance is unreachable and the correspondences hold vacuously.

These results are documented in archived summaries and raw output. The divergence package records its command, version, input hash, and zero exit status; it documents a fresh execution separately from the historical witness discussed in Chapter 5.⁴

⁴Evidence under `proofs/evidence/`: `authz-turepass-20260919T090826295518Z/` and `authz-chat-gates-20260918T023621017067Z/`.

6.6 Formalization Boundary

The matcher inconsistency is demonstrated at implementation level, and ProVerif models its protocol-level consequence under explicit assumptions. The models do not prove Python execution correctness, concrete rulebook evaluation, or a production deployment result. They establish reachability, blocking, and correspondences within the stated symbolic scenarios.

The prescribed peer processes do not represent arbitrary application behavior under attacker control. Private transport and shared state remain abstractions; token expiry, quota consumption, revocation, and policy updates are not covered. No cryptographic primitive failure or key compromise is established. The recorded correspondences are not a revalidation of every original authentication or secrecy property. A strict implementation-to-model mapping and service-level reproduction remain outside the completed evidence.

7 Evaluation

7.1 Experimental Setup

We evaluate three distinct questions using archived evidence: whether the matcher respects its documented policy semantics, whether the existing verification covers authorization, and what consequences follow from an explicit authorization divergence in the symbolic model. Implementation observations and formal results are reported separately. This evaluation introduces no additional experiments.

The matcher record uses Python 3.14.3 for direct function calls. The formal records use ProVerif 2.05 on Windows. Three evidence packages provide the inputs and results: Stage 1 (S), the divergence model (D), and independent gate controls (G).⁵ Their manifests record execution metadata, source revisions, commands, input hashes, and output provenance; raw standard output, standard error, and verification summaries accompany the runs. The reported model inputs match their recorded hashes. These archives originate from separate revisions, rather than a single integrated service experiment.

Table 2 reports the six communication-model runs used here. Each completed with exit status zero. Counts refer to final query results, not to satisfied security properties: reachability is reported through negated-event queries. Durations are single recorded verifier runs, rounded to one decimal place. They do not measure service overhead or establish a performance comparison; repeated measurements and a controlled hardware benchmark are outside this evaluation.

Package	Model / scenario	Results	Seconds
S	Original communication	11	334.6
S	Authorization diagnostic	12	353.7
D	Divergence (turepass)	2	598.4
G	Provider allow / Receiver allow	5	813.6
G	Provider deny / Receiver allow	5	507.1
G	Provider allow / Receiver deny	5	552.5

Table 2: Archived ProVerif runs used in the evaluation.

⁵Packages under `proofs/evidence/`: S = `authz-stage1-20260916T123834944479Z/`; D = `authz-turepass-20260919T090826295518Z/`; G = `authz-chat-gates-20260918T023621017067Z/`.

7.2 Matcher Evaluation

The intended semantics select the most specific matching rule. The self-contained `proofs/test_matcher_bug.py` illustrates the inconsistency with a specific deny rule for `mallory@example.com:*` followed by a wildcard allow rule. This explanatory script defines its own matcher variants and simplified specificity function; it does not import the implementation matcher.

Package S supplies direct implementation evidence in `policy-matcher-sanity.json`. Its recorded invocation uses `alice@example.com:agent`, a specific rule `alice@example.com:*` with budget `-1`, and a wildcard rule with budget `10`. The rulebook is valid, and the recorded specificity scores are `70` and `0`. The documented semantics therefore require budget `-1` in either ordering. Table 3 reports the saved results for these two orderings.

Rule order	Expected budget	Returned budget
Specific deny, then wildcard allow	-1 (deny)	10 (allow)
Wildcard allow, then specific deny	-1 (deny)	-1 (deny)

Table 3: Direct matcher observations for the recorded Alice rulebook.

The first ordering establishes a concrete policy decision mismatch; the second shows that reordering the same rules changes the result. This is an implementation-level, order-dependent authorization bypass. The record covers matcher calls, not Provider processing, token issuance, or message acceptance in a running service. Two observed orderings do not estimate the prevalence of the defect across rulebooks or deployments.

7.3 Formal Verification Results

Authorization coverage gap. The original communication model in S produces eleven results: seven event-reachability results, three authentication correspondences, and one token-secrecy result. The diagnostic model preserves these results and adds the authorization correspondence

$$\text{Accept}(R, I, t) \implies \text{PolicyAllow}(R, I).$$

This added correspondence is false, while the three authentication correspondences and token secrecy remain true. Here, **Accept** denotes receipt and verification of a token by the initiating peer; it is not application-message acceptance.

The diagnostic process never emits **PolicyAllow**. Consequently, the failed correspondence exposes an authorization obligation absent from the

original verification; it does not independently demonstrate a matcher execution error. The retained authentication and secrecy results apply to these Stage 1 queries. They are not automatically inherited by the later communication models.

Authorization consequence model. Package D contains two final results for `turepass`. The query asserting that `ChatAccept` is unreachable is false, with a reconstructed formal reachability witness. Under preset `ScenarioSpecDeny` and `ScenarioImplAllow` facts, the witness contains intended denial, material release, token issuance, and application-message acceptance. This is a symbolic reachability consequence of the divergence abstraction.

The second result establishes

$$\text{ChatAccept}(R, I, t, m) \Rightarrow \text{TokenIssue}(R, I, t).$$

Thus, acceptance can coexist with the assumed policy denial while still having corresponding prior token issuance. Credential provenance and policy permission remain separate obligations. Issuance is observed in the witness; this model has no standalone `TokenIssue` reachability query and no `ChatAccept`–`ChatSend` correspondence query. The complete D archive supplies these two final results with independent execution provenance, beyond the historical `turetrace` excerpt.

Authorization gate experiments. Package G evaluates three fixed pairs of Provider and receiver permissions. Each control has three reachability queries and two correspondences. Table 4 translates the reachability results into reachable (R) or unreachable (U), avoiding ambiguity about negated-query truth values.

Provider	Receiver	Release	Issue	Accept
Allow	Allow	R	R	R
Deny	Allow	U	U	U
Allow	Deny	R	U	U

Table 4: Gate-control reachability: `ProviderRelease`, `TokenIssue`, and `ChatAccept`.

Provider denial prevents release and downstream actions. Receiver denial prevents issuance and acceptance, while Provider release remains reachable. The allow/allow control establishes an executable permitted path.

Both acceptance-to-issuance and acceptance-to-send correspondences are true in all three controls. In the denial cases they hold vacuously, because acceptance is unreachable; only the allow/allow case supplies a non-vacuous control. These experiments locate blocking stages within the model, rather than evaluate a deployed repair. The controls use a fixed private token, unlike the fresh tokens in D, and are not a verified refinement of the divergence model.

7.4 Security Findings Summary

The evidence supports three complementary findings. First, direct matcher calls demonstrate a decision inconsistent with documented specificity semantics. Second, the coverage diagnostic distinguishes authorization from the authentication and secrecy properties actually queried in Stage 1. Third, given explicit divergence assumptions, the consequence model permits message acceptance, while the gate controls identify distinct release and issuance boundaries.

Together, these findings support the SAGA case study’s authorization enforcement gap. The implementation observation establishes the local defect; the formal results characterize modeled consequences and control positions. Neither evidence layer substitutes for the other. No result establishes cryptographic primitive failure, certificate forgery, key compromise, or arbitrary attacker control of the modeled peer processes.

7.5 Limitations

The formal model abstracts the matcher result. It neither executes Python nor evaluates the concrete rulebook, so the implementation record and the formal reachability witness are not a single executable reproduction. A strict semantic mapping from concrete rules, specificity, and returned budgets to scenario facts and formal events remains necessary.

Private transport and the `ProviderReleased` handoff are model abstractions. The evaluated queries do not establish token expiry, quota consumption, revocation, policy-update behavior, session isolation, or injective agreement. No complete production-service reproduction or deployment-scale measurement is reported. Future work may extend the semantic mapping and evaluate service behavior and larger deployments; the present results establish neither their outcomes nor generality across agentic systems.

8 Discussion

8.1 Why Authentication Proofs Are Insufficient

8.2 Formal Verification Coverage

8.3 Policy Semantics as Security-Critical State

8.4 Generalization Beyond SAGA

8.5 Limitations

9 Related Work

9.1 Agent Security and Governance

9.2 Capability and Token-Based Authorization

9.3 Formal Verification of Authentication Protocols

9.4 Formal Verification of Authorization Policies

9.5 Authentication–Authorization Gaps

10 Conclusion